This document synthesizes the business logic of CargoClear-JAPAC with the underlying GCP/ADK architecture, identifies critical compliance gaps in the provided engineering scaffold, and delivers a finalized, unified technical implementation plan.

1. Business to Architecture Translation

The CargoClear-JAPAC system translates macroeconomic trade friction into deterministic software engineering. The core objective is protecting SME Custom House Agents (CHAs) from demurrage charges and Section 149 customs holds triggered by documentation discrepancies.

- **Data Sovereignty & Security:** To comply with MeitY mandates for handling sovereign customs data, the entire infrastructure (Cloud Run, GCS, Firestore, BigQuery) is strictly pinned to the GCP asia-south1 (Mumbai) region.
- **e-SANCHIT Pre-Processing:** Indian customs mandates specific formats. Raw PDFs are intercepted and programmatically sanitized (binarized to 200 DPI black and white, deskewed, and split if exceeding 1MB or 13 pages) before LLM ingestion.
- **Adversarial Pre-Adjudication:** The system evaluates trade bundles via Google Agent Development Kit (ADK) pipelines to assign one of four hierarchical statuses: MATERIAL_CONTRADICTION > MISSING_EVIDENCE > SPECIALIST_REVIEW_REQUIRED > COMPLIANT_FOR_SUBMISSION.
- **Facts Before Interpretation (Deterministic Math):** LLMs are prohibited from doing math. Multimodal models (gemini-2.5-flash) extract spatial JSON. A pure-Python engine calculates weight parity (\sum Net + Tare == Gross within a 0.5% or 5kg tolerance) and semantic entity alignment.
- **Regulatory Grounding:** To prevent ATIGA Form D rejections, BigQuery VECTOR_SEARCH utilizes a TREE_AH index to semantically match declared items against the ASEAN Harmonized Tariff Nomenclature (AHTN) database and identify missing required documents.
- **Investigator UX:** The UI is an accessible (WCAG 2.2 AA) Next.js 15 shell. It overlays normalized bounding boxes onto HTML5 <canvas> elements and relies on Server-Sent Events (SSE) to prevent proxy timeouts during 45-second assessment runs.

2. Identified Blueprint Gaps & Resolutions

An audit of the provided scaffold ("Mvp draft_2_Technical implementation plan.txt") against the core specification ("CargoClear Dev Implementation Report.pdf") reveals four critical implementation gaps.

Gap A: Eventarc Decoupling Bypassed

- **Defect:** The provided blueprint executes file uploads and preprocessing synchronously via POST /bundles/{id}/documents. The original specification mandates an asynchronous, event-driven pipeline where Eventarc monitors GCS for the google.cloud.storage.object.v1.finalized event and triggers the backend via HTTP.
- **Resolution:** Remove synchronous preprocessing from the Next.js BFF proxy. Expose a dedicated POST /api/v1/webhooks/gcs endpoint on the FastAPI backend specifically for Eventarc invocations.

Gap B: Schema Omissions for Entity Alignment and SB003 Fatal Errors

- **Defect:** The specification requires semantic entity alignment for Consignee name, Vessel/Voyage number, Port of Loading, and invoice dates. Furthermore, it highlights ICEGATE SB003 (GSTIN mismatch) as a fatal error. The blueprint's ExtractedDocument Pydantic schema and calculation service omit port_of_loading, invoice_date, and gstin.
- **Resolution:** Expand the Pydantic domain contracts and update the pure-Python calculation engine to enforce these specific alignment checks.

Gap C: Missing Firestore Composite Index Configurations

- **Defect:** The UI must query assessment iterations (e.g., filtering by status and ordering chronologically). The specification explicitly notes that Firestore requires these to be defined in firestore.indexes.json. The blueprint repository layout omits this file.
- **Resolution:** Add firestore.indexes.json to the /infra root directory for deployment.

Gap D: High-Throughput Batch Processing Ignored

- **Defect:** The blueprint hardcodes synchronous gemini-2.5-flash extraction. The specification requires utilizing the Vertex AI Batch API with gemini-2.5-pro for overnight, asynchronous multi-bundle loads.
- **Resolution:** While Flash handles the interactive UI flow, the architectural router must support a batch mode flag that offloads payloads to a GCS JSONL queue for the Batch API.

3. Final Unified Implementation Blueprint

Below is the corrected, production-ready scaffold incorporating the resolutions above.

3.1 Corrected Repository Layout

```
cargoclear/
├── frontend/                        # Next.js 15 (TypeScript)
│   └── ... (matches original
scaffold)[span_19](start_span)[span_19](end_span)
├── backend/                         # FastAPI
│   ├── app/
│   │   ├── api/v1/
│   │   │   ├── webhooks.py      # FIXED: Eventarc ingestion endpoints
│   │   │   └── ...
│   │   └── ... (matches original
scaffold)[span_20](start_span)[span_20](end_span)
├── infra/
│   ├── firestore.indexes.json   # FIXED: Composite index
configurations[span_21](start_span)[span_21](end_span)
```

```
|   ├── bootstrap.sh
|   ├── bigquery/vector_index.sql
|   └── iam/roles.yaml
└── cloudbuild.yaml
```

3.2 Corrected Domain Contracts (Pydantic v2)

The schema is expanded to capture fields required for regulatory entity alignment and SB003 GSTIN mismatch detection.

```
# backend/app/domain/schemas.py
from typing import Literal
from pydantic import BaseModel, Field

class BBox(BaseModel):
    page: int = Field(ge=1)
    ymin: float = Field(ge=0, le=1000)
    xmin: float = Field(ge=0, le=1000)
    ymax: float = Field(ge=0, le=1000)
    xmax: float = Field(ge=0, le=1000)

class FieldRef(BaseModel):
    value: str | float | int | None = None
    confidence: float = Field(ge=0.0, le=1.0)
    bbox: BBox | None = None

class ExtractedDocument(BaseModel):
    document_id: str
    doc_type: Literal["COMMERCIAL_INVOICE", "BILL_OF_LADING",
"PACKING_LIST", "CERTIFICATE_OF_ORIGIN", "UNKNOWN"]
    doc_type_confidence: float = Field(ge=0.0, le=1.0)

    # Core Entities
    document_number: FieldRef | None = None
    invoice_date: FieldRef | None = None          # FIXED: Added for
alignment
    gstin: FieldRef | None = None                 # FIXED: Required
for SB003 checks

    # Shipping Entities
    consignee: FieldRef | None = None
    consignor: FieldRef | None = None
    vessel_or_flight: FieldRef | None = None
    port_of_loading: FieldRef | None = None       # FIXED: Added for
alignment

    # Financials & Weights
    total_net_weight_kg: FieldRef | None = None
```

```
    total_gross_weight_kg: FieldRef | None = None
    tare_weight_kg: FieldRef | None = None
    line_items: list['LineItem'] = []
```

3.3 Eventarc Integration (Gateway Layer)

Direct GCS uploads bypass the FastAPI processing layer. Eventarc must push metadata to the webhook to trigger preprocessing asynchronously.

```
# backend/app/api/v1/webhooks.py
from fastapi import APIRouter, Request, BackgroundTasks
from app.services import preprocess, orchestrator

router = APIRouter()

@router.post("/webhooks/gcs")
async def gcs_eventarc_trigger(request: Request, bg_tasks:
BackgroundTasks):
    """Triggered by google.cloud.storage.object.v1.finalized"""
    payload = await request.json()
    bucket = payload.get("bucket")
    name = payload.get("name") # e.g.,
bundles/123/raw/doc_456/original.pdf

    if "original.pdf" in name:
        bundle_id = name.split("/")[1]
        document_id = name.split("/")[3]

        # Enqueue e-SANCHIT
pre-processing[span_24](start_span)[span_24](end_span)
        bg_tasks.add_task(preprocess.run_async, bucket, name,
bundle_id, document_id)

    return {"status": "accepted"}
```

3.4 Corrected Deterministic Calculation Engine

Updates include exact matching rules for the newly added schema fields.

```
# backend/app/services/calculation.py

def check_entity_alignment(docs: list[ExtractedDocument]) ->
list[CheckResult]:
    """Consignee / vessel / port / date / GSTIN alignment."""
    out: list[CheckResult] = []

    # FIXED: Expanded alignment fields matching Section 149
triggers[span_26](start_span)[span_26](end_span)
```

```
    critical_fields = ("consignee", "vessel_or_flight",
"port_of_loading", "invoice_date", "gstin")

    for field in critical_fields:
        vals = {getattr(d, field).value.strip().upper() for d in docs
if getattr(d, field) and getattr(d, field).value}

        if len(vals) > 1:
            severity = "MATERIAL"
            msg = f"Inconsistent {field}: {sorted(map(str, vals))}"

            # FIXED: Explicitly flag SB003 fatal
errors[span_27](start_span)[span_27](end_span)
            if field == "gstin":
                severity = "FATAL"
                msg = f"SB003 GSTIN Mismatch detected across bundle.
ICEGATE rejection imminent. Found: {sorted(map(str, vals))}"

            out.append(CheckResult(f"E-{field.upper()}", severity,
msg))

    return out
```

3.5 Firestore Composite Indexing

Required to support the chronological timeline filtering in the UI without failing at scale.

```
// infra/firestore.indexes.json
{
  "indexes": [
    {
      "collectionGroup": "iterations",
      "queryScope": "COLLECTION",
      "fields": [
        {
          "fieldPath": "bundle_id",
          "order": "ASCENDING"
        },
        {
          "fieldPath": "status",
          "order": "ASCENDING"
        },
        {
          "fieldPath": "created_at",
          "order": "DESCENDING"
        }
      ]
    }
```

```
    ]
}
```